Tab 1

Mini project report on

Operating Systems

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

NANDAN C	**PES2UG24CS301**
MUTHURAJ S M	**PES2UG24C289**

under the guidance of

Dr. Chithambarathanu M

Assistant Professor

PES University

Jan 2026 - May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Operating Systems

is a bona fide work carried out by

NANDAN C	**PES2UG24CS301**
MUTHURAJ S M	**PES2UG24CS289**

In partial fulfilment for the completion of the fourth semester **OS Project (UE24CS242B)** in the Program of Study - **Bachelor of Technology** in **Computer Science and Engineering** under the rules and regulations of **PES University**, Bengaluru, during the period **Jan 2026 – May 2026**. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the fourth semester academic requirements with respect to project work.

Signature

Dr. Chithambarathanu M

Assistant Professor

DECLARATION

We hereby declare that the Operating Systems Project entitled **Scheduler Experiments and Analysis** has been carried out by us under the guidance of **Dr. Chithambarathanu M, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University**, Bengaluru, during the academic semester **Jan 2026 – May 2026**.

NANDAN C	**PES2UG24CS301**
MUTHURAJ S M	**PES2UG24CS289**

ACKNOWLEDGEMENT

I would like to express my gratitude to **Dr. Chithambarathanu M, Department of Computer Science and Engineering, PES University**, for his continuous guidance, assistance, and encouragement throughout the development of this **UE24CS242B - Operating Systems Project**.

I take this opportunity to thank **Dr. Sandesh B J, Professor, Chairperson, Department of Computer Science and Engineering, PES University**, for all the knowledge and support I have received from the department.

I am deeply grateful to **Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University**, for providing me with various opportunities and enlightenment every step of the way. Finally, this OS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

This project presents the design and implementation of a lightweight multi-container runtime in C on a Linux environment, integrating both user-space and kernel-space components. The system includes a long-running supervisor process responsible for managing multiple isolated containers concurrently, along with a kernel module that monitors and enforces memory usage policies.

Each container is created using Linux namespaces (PID, UTS, and mount) and operates within its own isolated root filesystem derived from a minimal Alpine Linux base. The supervisor provides a command-line interface (CLI) that allows users to start, stop, monitor, and inspect containers through an inter-process communication (IPC) mechanism. Container outputs are captured using a bounded-buffer logging system, ensuring safe concurrent access and persistent storage of logs.

In addition, a custom Linux kernel module is developed to track memory usage per container using the /proc filesystem, triggering configurable soft and hard memory limits. The project demonstrates practical understanding of Linux system programming, including process isolation, signal handling, IPC, concurrent synchronization, and kernel module development.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	1
2.	PROBLEM DEFINITION	2
3.	METHODOLOGY	3
4.	CODE	4
5.	RESULT AND DISCUSSION	7
6.	OUTPUTS	10
7.	GITHUB LINK	11

Tab 2

INTRODUCTION

This project implements a Supervisor–CLI architecture for managing container-like processes in a Linux environment. The system mimics a simplified version of modern container runtimes such as Docker by separating control logic (CLI) from execution logic (supervisor daemon).

The supervisor is a long-running process responsible for:

- ***Creating and managing containers (child processes)***
- ***Tracking process state***
- ***Handling system signals***
- ***Managing logs***

The CLI acts as a client interface that sends commands to the supervisor via Inter-Process Communication (IPC).

This design ensures modularity, scalability, and clean separation of concerns between user interaction and process management.

PROBLEM DEFINITION

Task 2: Supervisor CLI and Signal Handling

Implement a CLI interface for interacting with the supervisor.

The command grammar and semantics in Canonical CLI Contract (Required) are mandatory.

Required commands:

- ***start – to launch a new container in the background***
- ***run – to launch a container and wait for it in the foreground***
- ***ps – to list tracked containers and their metadata***
- ***logs – to inspect a container log file***
- ***stop – to terminate a running container cleanly***

You may add more commands if you want, but the above are required.

Demonstrate:

- ***CLI requests reach the long-running supervisor correctly***
- ***Supervisor updates container state after each command***
- ***SIGCHLD handling is correct and does not leak zombies***
- ***SIGINT/SIGTERM to the supervisor trigger orderly shutdown***
- ***Container termination path distinguishes graceful stop vs forced kill***

This task covers Path B (control): the IPC channel between the CLI client process and the supervisor daemon (see Architecture Overview). This channel must use a different IPC mechanism than the logging pipes in Task 3. A UNIX domain socket, FIFO, or shared-memory-based command channel are all acceptable if justified.

The CLI process sends a command string (e.g., start alpha ./rootfs-alpha /bin/sh --soft-mib 48 --hard-mib 80) over this channel. The supervisor reads the command, acts on it, and sends a response back. Design the message format so the supervisor can parse the command type, required arguments, and optional flags.

METHODOLOGY

3.1 Architecture

The system follows a client-server model:

- ***CLI → Sends commands***
- ***Supervisor → Executes and manages processes***

Communication is implemented using UNIX domain sockets, enabling efficient local IPC.

3.2 Process Management

Each container is implemented as a child process using:

- ***fork()***
- ***exec()***

The supervisor maintains metadata:

- ***Container name***
- ***Process ID (PID)***
- ***State (RUNNING / EXITED)***

3.3 Signal Handling

`SIGCHLD`

- ***Triggered when a child process exits***
- ***Handled using waitpid(..., WNOHANG)***
- ***Prevents zombie processes***

`SIGINT / SIGTERM`

- ***Used for graceful shutdown of supervisor***
- ***Ensures all running containers are terminated***

3.4 IPC Mechanism

UNIX domain sockets are used because:

- ***Fast and reliable***
- ***Bidirectional communication***
- ***Suitable for local systems***

CODE

Supervisor:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <sys/wait.h>
#include <fcntl.h>

#define SOCKET_PATH "/tmp/supervisor.sock"
#define MAX_CONTAINERS 100

typedef struct {
    char name[50];
    pid_t pid;
    int running;
} Container;

Container containers[MAX_CONTAINERS];
int count = 0;
int server_fd;

// Utility
int find_container(const char *name) {
    for (int i = 0; i < count; i++) {
        if (strcmp(containers[i].name, name) == 0)
            return i;
    }
    return -1;
}

// SIGCHLD
void sigchld_handler(int sig) {
    int status;
    pid_t pid;
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        for (int i = 0; i < count; i++) {
            if (containers[i].pid == pid) {
                containers[i].running = 0;
                break;
            }
        }
```

```
    }
}

// Shutdown
void cleanup(int sig) {
    printf("\n[Supervisor] Shutting down...\n");
    for (int i = 0; i < count; i++) {
        if (containers[i].running) {
            kill(containers[i].pid, SIGTERM);
        }
    }
    close(server_fd);
    unlink(SOCKET_PATH);
    exit(0);
}

// Start
void handle_start(int client, char *name, char *cmd) {
    pid_t pid = fork();
    if (pid == 0) {
        // Child -> redirect logs
        char logfile[100];
        sprintf(logfile, "%s.log", name);
        int fd = open(logfile, O_CREAT | O_WRONLY | O_APPEND,
   0644);
        dup2(fd, STDOUT_FILENO);
        dup2(fd, STDERR_FILENO);
        execl("/bin/sh", "sh", "-c", cmd, NULL);
        exit(1);
    }
    strcpy(containers[count].name, name);
    containers[count].pid = pid;
    containers[count].running = 1;
    count++;
    write(client, "STARTED\n", 8);
}

// Run (foreground)
void handle_run(int client, char *name, char *cmd) {
    pid_t pid = fork();
    if (pid == 0) {
        execl("/bin/sh", "sh", "-c", cmd, NULL);
        exit(1);
    }
    waitpid(pid, NULL, 0);
    write(client, "FINISHED\n", 9);
}

// PS
```

```
void handle_ps(int client) {
    char buffer[1024] = "";
    for (int i = 0; i < count; i++) {
        char line[128];
        sprintf(line, "%s | PID=%d | %s\n",
            containers[i].name,
            containers[i].pid,
            containers[i].running ? "RUNNING" : "EXITED");
        strcat(buffer, line);
    }
    write(client, buffer, strlen(buffer));
}

// STOP
void handle_stop(int client, char *name) {
    int idx = find_container(name);
    if (idx == -1) {
        write(client, "NOT FOUND\n", 10);
        return;
    }
    pid_t pid = containers[idx].pid;
    kill(pid, SIGTERM);
    sleep(1);
    if (kill(pid, 0) == 0) {
        kill(pid, SIGKILL);
        write(client, "FORCE KILLED\n", 13);
    } else {
        write(client, "GRACEFUL STOP\n", 14);
    }
}

// LOGS
void handle_logs(int client, char *name) {
    char filename[100];
    sprintf(filename, "%s.log", name);
    FILE *f = fopen(filename, "r");
    if (!f) {
        write(client, "NO LOGS\n", 8);
        return;
    }
    char buffer[256];
    while (fgets(buffer, sizeof(buffer), f)) {
        write(client, buffer, strlen(buffer));
    }
    fclose(f);
}

// Command Dispatcher
void process_command(int client, char *cmd) {
```

```
    char *token = strtok(cmd, " \n");
    if (!token) return;
    if (strcmp(token, "start") == 0) {
        char *name = strtok(NULL, " ");
        char *rest = strtok(NULL, "\n");
        handle_start(client, name, rest);
    }
    else if (strcmp(token, "run") == 0) {
        char *name = strtok(NULL, " ");
        char *rest = strtok(NULL, "\n");
        handle_run(client, name, rest);
    }
    else if (strcmp(token, "ps") == 0) {
        handle_ps(client);
    }
    else if (strcmp(token, "stop") == 0) {
        char *name = strtok(NULL, " ");
        handle_stop(client, name);
    }
    else if (strcmp(token, "logs") == 0) {
        char *name = strtok(NULL, " ");
        handle_logs(client, name);
    }
    else {
        write(client, "UNKNOWN COMMAND\n", 16);
    }
}

// Main
int main() {
    struct sockaddr_un addr;
    signal(SIGCHLD, sigchld_handler);
    signal(SIGINT, cleanup);
    signal(SIGTERM, cleanup);
    server_fd = socket(AF_UNIX, SOCK_STREAM, 0);
    unlink(SOCKET_PATH);
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strcpy(addr.sun_path, SOCKET_PATH);
    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 5);
    printf("[Supervisor] Running...\n");
    while (1) {
        int client = accept(server_fd, NULL, NULL);
        char buffer[1024] = {0};
        read(client, buffer, sizeof(buffer));
        process_command(client, buffer);
        close(client);
    }
```

```
    return 0;
}
```

CLI Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

#define SOCKET_PATH "/tmp/supervisor.sock"

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: ./cli <command>\n");
        return 1;
    }
    int sock = socket(AF_UNIX, SOCK_STREAM, 0);
    struct sockaddr_un addr;
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strcpy(addr.sun_path, SOCKET_PATH);
    if (connect(sock, (struct sockaddr*)&addr, sizeof(addr)) == -1) {
        perror("Connect failed");
        return 1;
    }
    // Build command string
    char command[1024] = "";
    for (int i = 1; i < argc; i++) {
        strcat(command, argv[i]);
        strcat(command, " ");
    }
    write(sock, command, strlen(command));
    char buffer[1024];
    int n;
    while ((n = read(sock, buffer, sizeof(buffer)-1)) > 0) {
        buffer[n] = '\0';
        printf("%s", buffer);
    }
    close(sock);
    return 0;
}
```

RESULT AND DISCUSSION

The implemented Supervisor–CLI system successfully demonstrates reliable process management using a client–server architecture over UNIX domain sockets. The CLI effectively communicates with the long-running supervisor, and all commands such as start, run, ps, stop, and logs function as intended. Containers are created as child processes, and their states transition correctly from RUNNING to EXITED, which is accurately reflected in the ps output. The logging mechanism captures container outputs and allows later inspection. Signal handling is robust—SIGCHLD is properly managed using non-blocking waitpid, ensuring that no zombie processes are left behind, while SIGINT and SIGTERM enable clean shutdown of the supervisor by terminating all active containers. The termination logic correctly distinguishes between graceful stopping (via SIGTERM) and forced killing (via SIGKILL) when necessary. Overall, the system performs efficiently with minimal overhead due to lightweight process-based containers, though it lacks advanced features such as resource isolation and structured communication protocols, which could be considered for future improvements.

OUTPUTS

```
nandan-c@nandan-c-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine supervisor
../rootfs-base
[Supervisor] Starting...
[Supervisor] Started container alpha with PID 13567
[Supervisor] Started container beta with PID 13568
/ # / #
```

```
nandan-c@nandan-c-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine supervi-
sor
[Supervisor] Running...
[Supervisor] cmd=2 id=
[Supervisor] cmd=1 id=alpha
/ # [Supervisor] cmd=2 id=
[Supervisor] cmd=3 id=alpha
[Supervisor] cmd=2 id=
```

```
nandan-c@nandan-c-VirtualBox:~/Documents/OS_mini$ gcc cli.c -o cli
nandan-c@nandan-c-VirtualBox:~/Documents/OS_mini$ ./cli
Usage: ./cli <command>
nandan-c@nandan-c-VirtualBox:~/Documents/OS_mini$ ./cli start alpha "sleep 10"
STARTED
./cli ps
nandan-c@nandan-c-VirtualBox:~/Documents/OS_mini$ ./cli ps
alpha | PID=57947 | EXITED
nandan-c@nandan-c-VirtualBox:~/Documents/OS_mini$ ./cli stop alpha
GRACEFUL STOP
nandan-c@nandan-c-VirtualBox:~/Documents/OS_mini$ ./cli logs alpha
nandan-c@nandan-c-VirtualBox:~/Documents/OS_mini$ ./cli run test "ls -l"
FINISHED
nandan-c@nandan-c-VirtualBox:~/Documents/OS_mini$
```

```
nandan-c@nandan-c-VirtualBox:~/Documents/OS_mini$ gcc supervisor.c -o supervisor
nandan-c@nandan-c-VirtualBox:~/Documents/OS_mini$ ./supervisor
[Supervisor] Running...
total 52
-rw-r--r-- 1 nandan nandan      0 Apr 14 18:28 alpha.log
-rwxrwxr-x 1 nandan nandan  16440 Apr 14 18:27 cli
-rw-rw-r-- 1 nandan nandan   1009 Apr 14 18:25 cli.c
-rwxrwxr-x 1 nandan nandan  17552 Apr 14 18:26 supervisor
-rw-rw-r-- 1 nandan nandan   5171 Apr 14 18:24 supervisor.c
```

```
nandan-c@nandan-c-VirtualBox:/media/psf/Home/Desktop/OS/OS-Jackfruit/boilerplate$
sudo ./engine ps
ID        PID      STATE    STARTED
beta      26564    exited   1776141009
alpha     26556    exited   1776141009
```

```
nandan-c@nandan-c-VirtualBox:/media/psf/Home/Desktop/OS/OS-Jackfruit/boilerplate$ sudo ./engine logs alpha
Inside container! Setting up environment...
Container ready. Running command: /bin/sh
execve: Exec format error
Inside container! Setting up environment...
Container ready. Running command: /bin/sh
Inside container! Setting up environment...
Container ready. Running command: /memory_hog
allocation=1 chunk=8MB total=8MB
allocation=2 chunk=8MB total=16MB
allocation=3 chunk=8MB total=24MB
allocation=4 chunk=8MB total=32MB
allocation=5 chunk=8MB total=40MB
allocation=6 chunk=8MB total=48MB
allocation=7 chunk=8MB total=56MB
allocation=8 chunk=8MB total=64MB
Inside container! Setting up environment...
Container ready. Running command: /bin/busybox
BusyBox v1.36.1 (2024-06-10 07:11:47 UTC) multi-call binary.
BusyBox is copyrighted by many authors between 1998-2015.
Licensed under GPLv2. See source distribution for detailed
copyright notices.
```

```
nandan-c@nandan-c-VirtualBox:/media/psf/Home/Desktop/OS/OS-Jackfruit/boilerplate$ sudo ./engine start gamma
./rootfs /bin/busybox
Started gamma (PID 27957)
```

```
nandan-c@nandan-c-VirtualBox:/media/psf/Home/Desktop/OS/OS-Jackfruit/boilerplate$ sudo ./engine run memtest
./rootfs /memory_hog
Started memtest (PID 28209)
```

```
nandan-c@nandan-c-VirtualBox:/media/psf/Home/Desktop/OS/OS-Jackfruit/boilerplate$
sudo dmesg | tail -15
[ 4511.861005] [container_monitor] Registering container=alpha pid=25629
soft=41943040 hard=67108864
[ 4511.882419] [container_monitor] Unregister request container=alpha pid=25629
[ 4518.461226] [container_monitor] Registering container=beta pid=25673
soft=41943040 hard=67108864
[ 4518.504572] [container_monitor] Unregister request container=beta pid=25673
[ 4692.549146] [container_monitor] Module unloaded.
[ 4719.805495] [container_monitor] Module loaded. Device: /dev/container_monitor
[ 4743.067901] [container_monitor] Registering container=alpha pid=26556
soft=41943040 hard=67108864
[ 4743.094020] [container_monitor] Unregister request container=alpha pid=26556
[ 4743.094423] [container_monitor] Registering container=beta pid=26564
soft=41943040 hard=67108864
[ 4743.103603] [container_monitor] Unregister request container=beta pid=26564
[ 5119.412185] [container_monitor] Registering container=gamma pid=27957
soft=41943040 hard=67108864
[ 5119.442323] [container_monitor] Unregister request container=gamma pid=27957
[ 5183.000509] [container_monitor] Registering container=memtest pid=28209
soft=41943040 hard=67108864
[ 5187.810857] [container_monitor] SOFT LIMIT container=memtest pid=28209
rss=42119168 limit=41943040
[ 5190.885491] [container_monitor] HARD LIMIT container=memtest pid=28209
rss=67284992 limit=67108864
```

```
nandan-c@nandan-c-VirtualBox:/media/psf/Home/Desktop/OS/OS-Jackfruit/boilerplate$
time sudo ./engine run c1 ./rootfs /cpu_hog --nice 0 &
time sudo ./engine run c2 ./rootfs /cpu_hog --nice 10 &
wait
[1] 28482
[2] 28484
```

```
Started c1 (PID 28490)
Started c2 (PID 28534)
Container c1 finished

real    0m10.079s
user    0m0.007s
sys     0m0.019s
[1]- Done         time sudo ./engine run c1 ./rootfs /cpu_hog --nice 0
Container c2 finished

real    0m20.077s
user    0m0.007s
sys     0m0.013s
[2]+ Done         time sudo ./engine run c2 ./rootfs /cpu_hog --nice 10
```

```
nandan-c@nandan-c-VirtualBox:/media/psf/Home/Desktop/OS/OS-Jackfruit/boilerplate$
sudo ./engine stop alpha
sudo ./engine stop beta
sudo ./engine ps
ps aux | grep engine
Stopped alpha
Stopped beta
ID        PID     STATE   STARTED
c2        28534   exited  1776141529
c1        28490   exited  1776141519
memtest   28209   killed  1776141449
gamma     27957   exited  1776141385
beta      26564   stopped 1776141009
alpha     26556   stopped 1776141009
paralle+  3520  0.1  0.1 237728  7312 ?    Sl   08:41   0:08 /usr/libexec/ibus-
engine-simple
root      26461  0.0  0.1  20864  6308 pts/2  S+   09:59   0:00 sudo ./engine
supervisor ./rootfs
root      26465  0.0  0.0  20864  2468 pts/1  Ss   09:59   0:00 sudo ./engine
supervisor ./rootfs
root      26466  0.2  0.0  93628  1804 pts/1  Sl+ 09:59   0:01 ./engine
supervisor ./rootfs
paralle+ 28693  0.0  0.0   8752  2084 pts/0  S+   10:09   0:00 grep --color=auto
engine

^CSupervisor shutting down...
Supervisor exited cleanly.
```

```
nandan-c@nandan-c-VirtualBox:/media/psf/Home/Desktop/OS/OS-Jackfruit/boilerplate$
sudo rmmod monitor
sudo dmesg | tail -5
[ 5252.597244] [container_monitor] Registering container=c1 pid=28490
soft=41943040 hard=67108864
[ 5253.408875] [container_monitor] Process exited, removing container=c1 pid=28490
[ 5262.588267] [container_monitor] Registering container=c2 pid=28534
soft=41943040 hard=67108864
[ 5263.648860] [container_monitor] Process exited, removing container=c2 pid=28534
[ 5374.999179] [container_monitor] Module unloaded.
```

GITHUB LINK

- https://github.com/nandan-chiru/OS-Jackfruit-miniproject